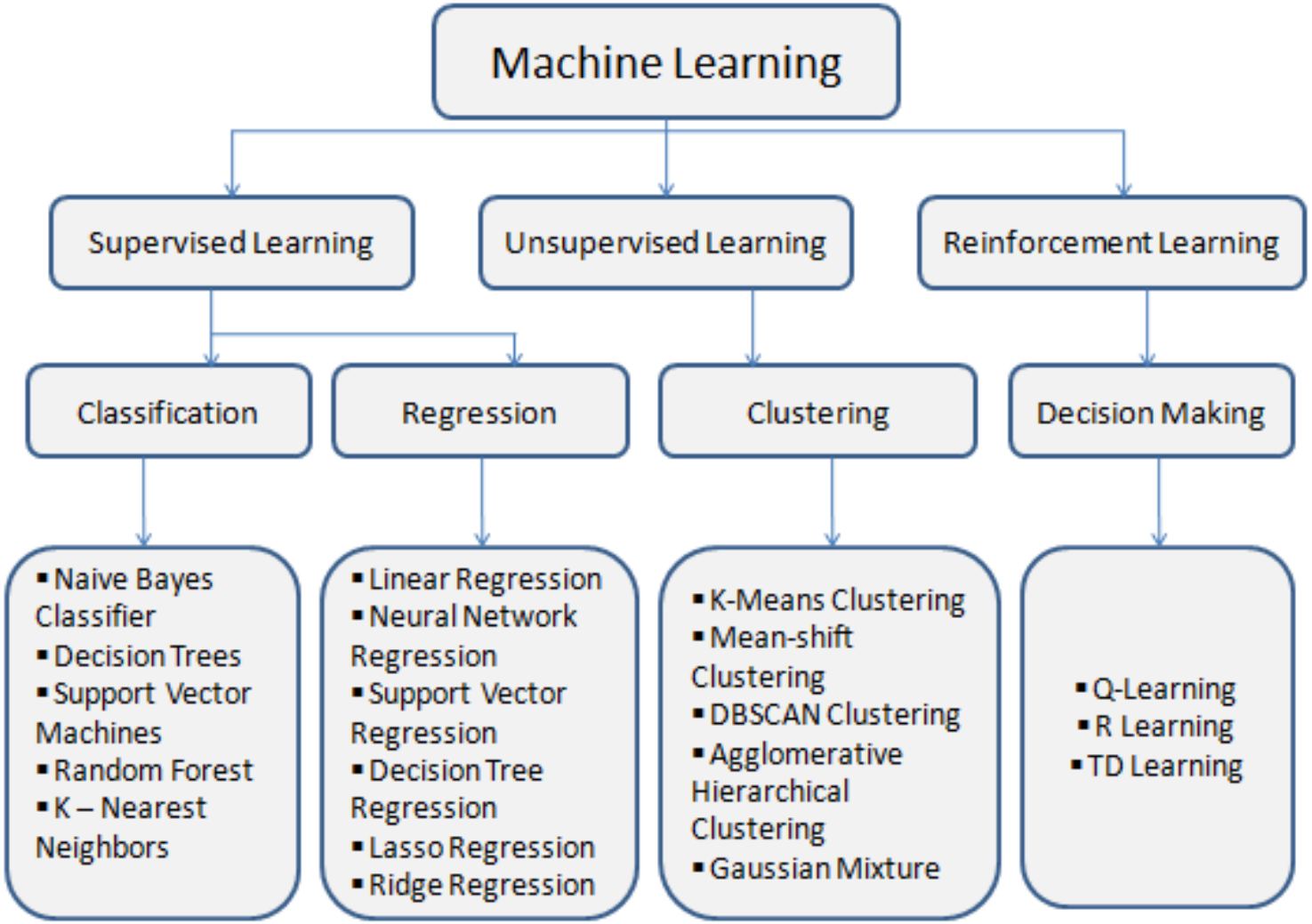
TRƯỜNG ĐẠI HỌC NGÂN HÀNG
THÀNH PHỐ HỒ CHÍ MINH

KHAI PHÁ DỮ LIỆU

CHƯƠNG 4
MỘT SỐ PHƯƠNG PHÁP PHÂN LỚP VÀ DỰ BÁO
(PHẦN 4)


H U B
1976
TRƯỜNG ĐẠI HỌC NGÂN HÀNG
THÀNH PHỐ HỒ CHÍ MINH

Xác định mục đích khai phá dữ liệu



Nội dung

1. Giới thiệu về phân lớp dữ liệu
2. Phân lớp dữ liệu với cây quyết định
3. Phương pháp phân lớp dữ liệu Bayes
4. Phân lớp và dự báo với máy vector hỗ trợ
5. Phân lớp và dự báo dữ liệu với mạng nơ-ron
6. Phân lớp k-NN
7. Hồi qui tuyến tính và hồi qui Logistic
8. Đánh giá và lựa chọn mô hình
9. Một số ứng dụng của phân lớp dữ liệu

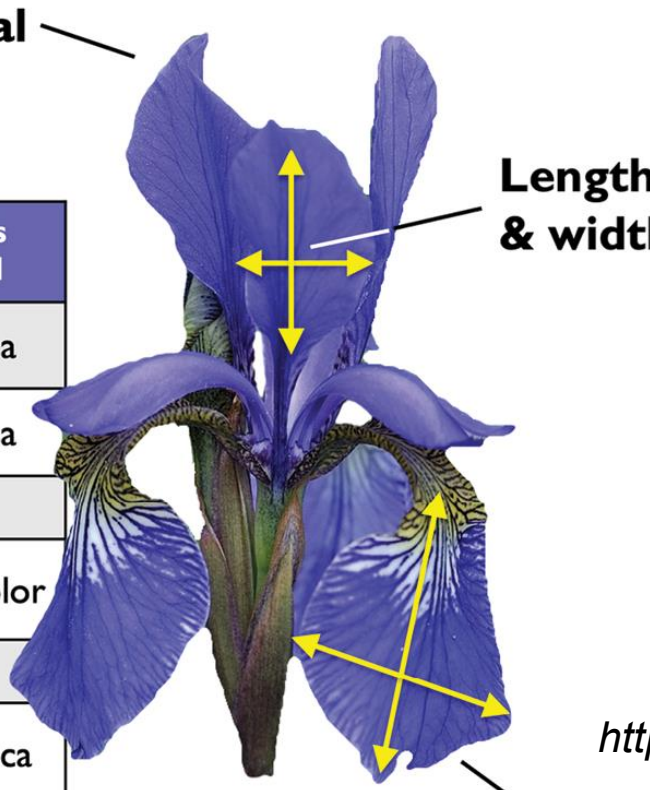
Phân lớp (Classification)

Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
			...		
50	6.4	3.5	4.5	1.2	Versicolor
			...		
150	5.9	3.0	5.0	1.8	Virginica

Features
(inputs, attributes, measurements, dimensions)

Class labels
(targets, outcomes)



Tập dữ liệu Iris dự đoán loài hoa dựa vào kích thước cánh hóa và đài hoa

https://archive.ics.uci.edu/ml/datasets/iris

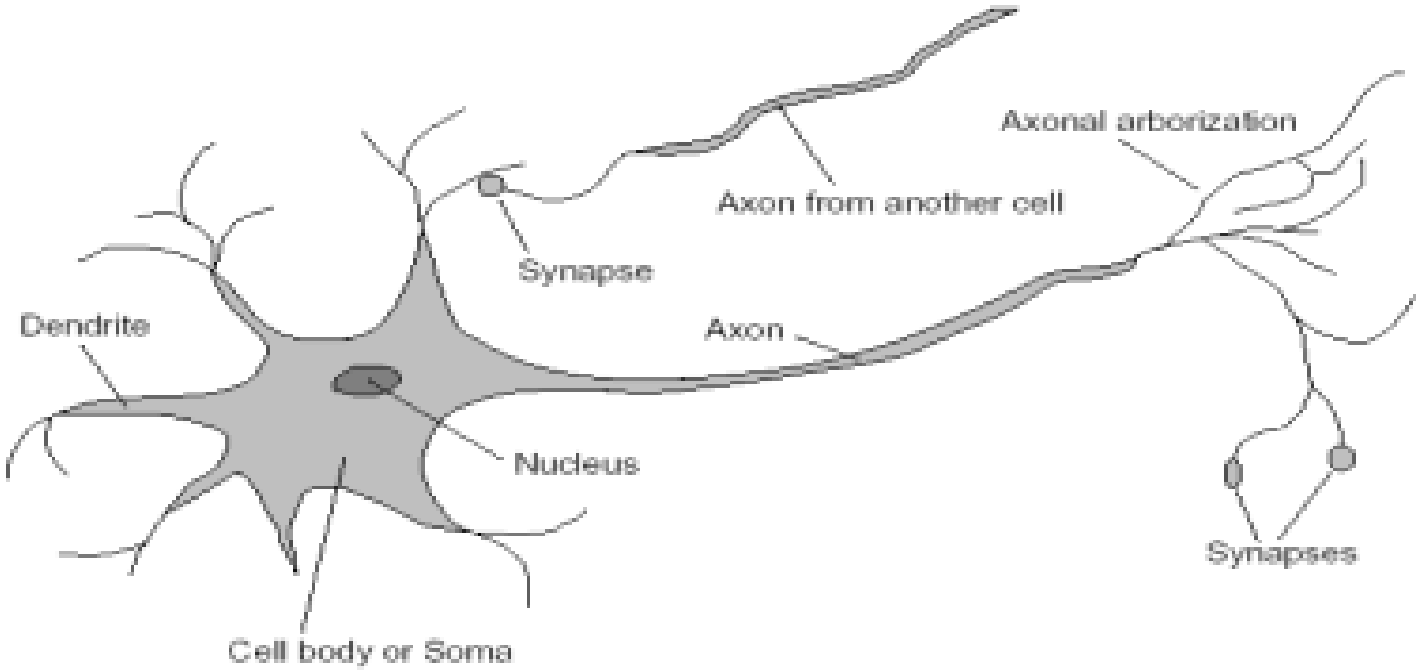
Hồi quy (Regression)

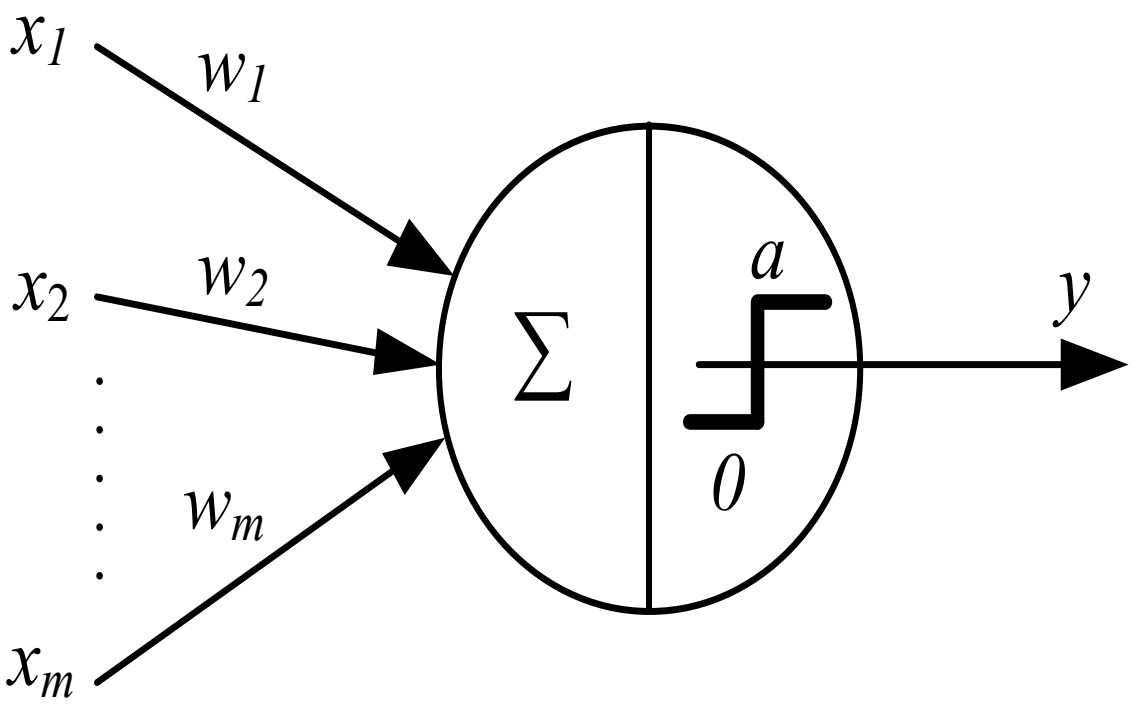
	A	B	C	D	E
1	area	bedrooms	balcony	age	price
2	1200	2	0	2	500000
3	2300	3	2	5	620000
4	2500	4	2	1	122500
5	3650	5	3	3	6000000
6	1800	3	1	5	2122000
7	3000	3	1	4	120000
8	1222	1	0	2	450000
9	4600	5	3	1	6500000
10	2050	2	2	2	1530000
11	1450	2	2	3	1563330

Nhãn liên tục

4. Phân lớp và dự báo dữ liệu với mạng nơ-ron

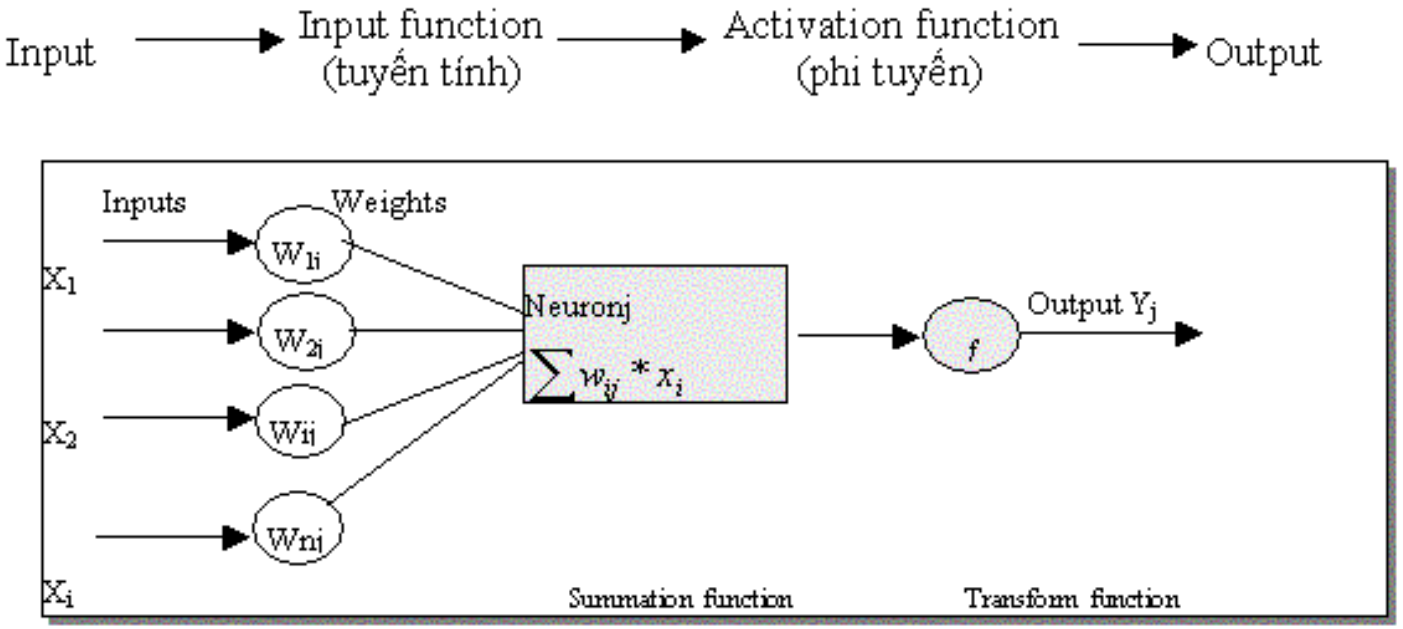
- Mạng Neural sinh học

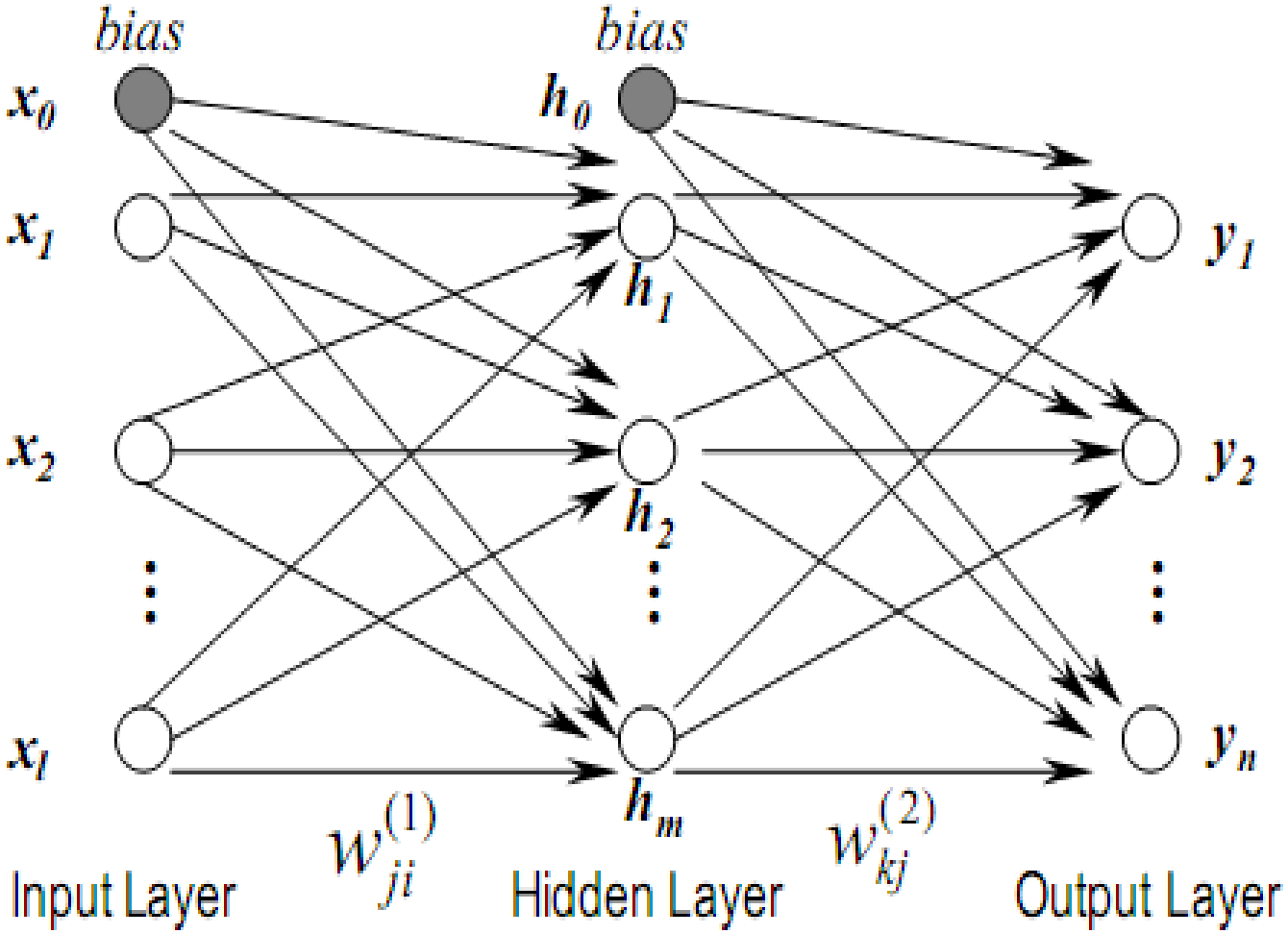



x_1
w_1
x_2
w_2
w_m
x_m
Σ
a
0
y

4.3 Mạng Neural

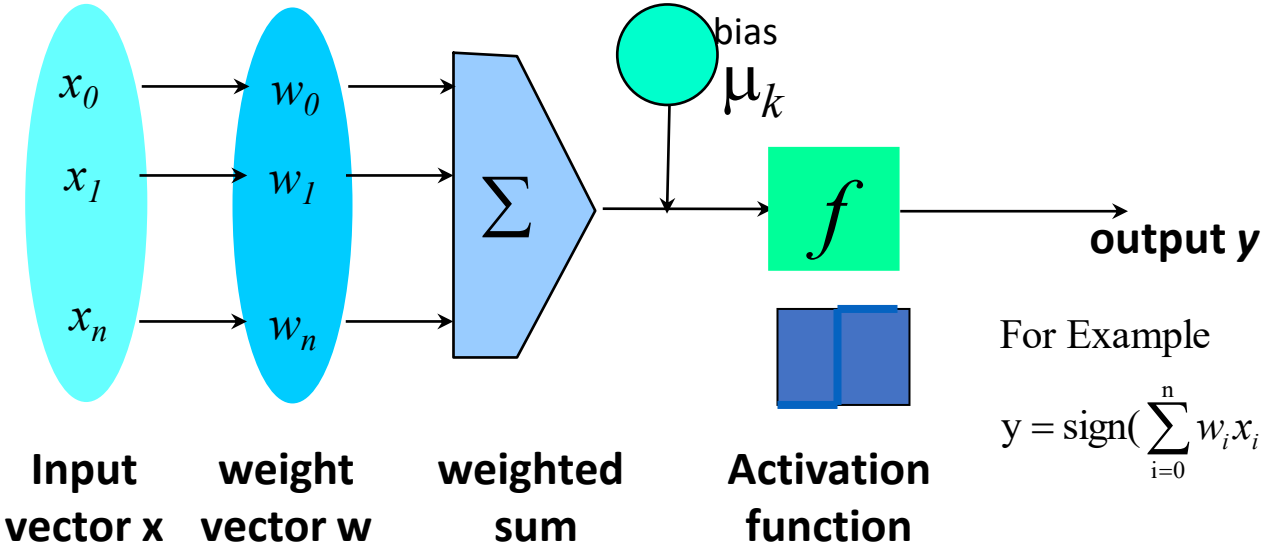
Quá trình xử lý thông tin tại một neuron của mạng Neural nhân tạo




bias
bias
x_0
x_1
x_2
x_I
h_0
h_1
h_2
h_m
y_1
y_2
y_n
$w_{ji}^{(1)}$
$w_{kj}^{(2)}$
Input Layer
Hidden Layer
Output Layer

Neuron: A Hidden/Output Layer Unit

- An n-dimensional input vector x is mapped into variable y by means of the scalar product and a nonlinear function mapping
- The inputs to unit are outputs from the previous layer. They are multiplied by their corresponding weights to form a weighted sum, which is added to the bias associated with unit. Then a nonlinear activation function is applied to it.



For Example

$$y = \text{sign}\left(\sum_{i=0}^{n} w_i x_i - \mu_k\right)$$

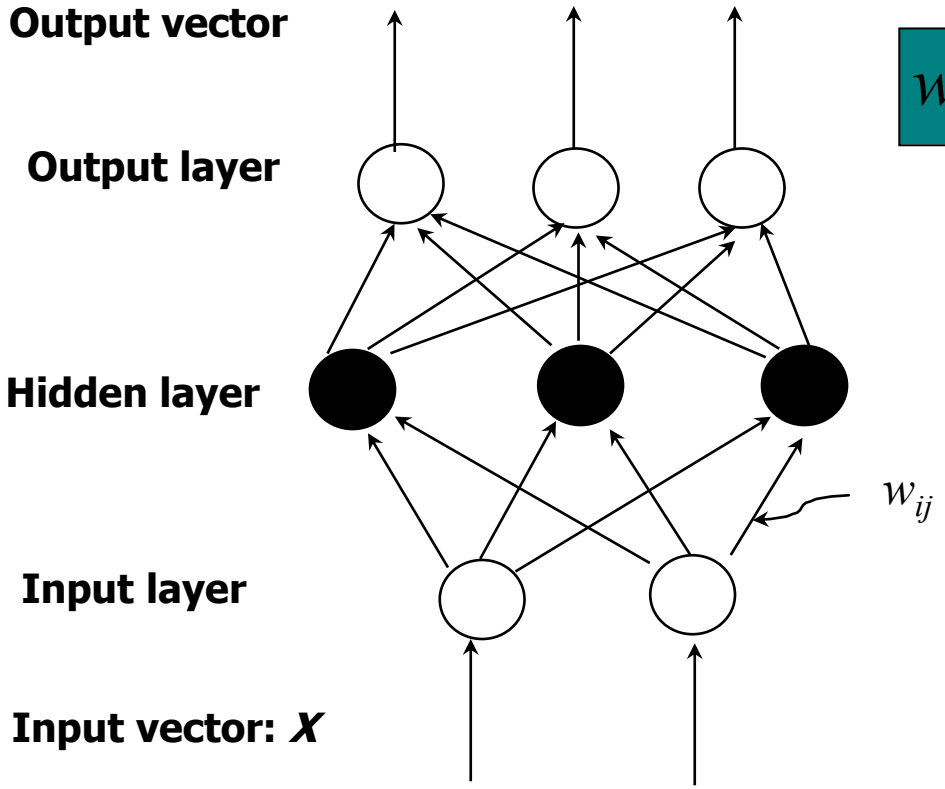
How A Multi-Layer Neural Network Works

- The inputs to the network correspond to the attributes measured for each training tuple
- Inputs are fed simultaneously into the units making up the input layer
- They are then weighted and fed simultaneously to a hidden layer
- The number of hidden layers is arbitrary, although usually only one
- The weighted outputs of the last hidden layer are input to units making up the output layer, which emits the network's prediction
- The network is feed-forward: None of the weights cycles back to an input unit or to an output unit of a previous layer
- From a statistical point of view, networks perform nonlinear regression: Given enough hidden units and enough training samples, they can closely approximate any function

Defining a Network Topology

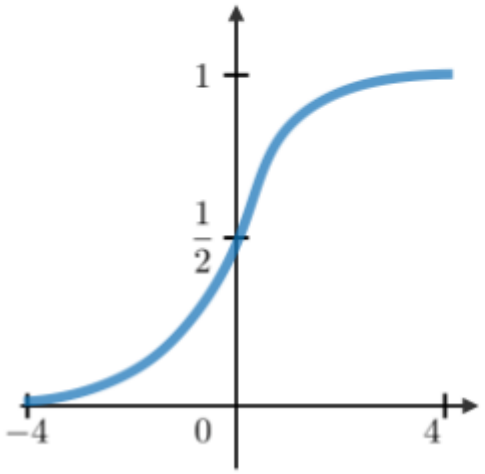
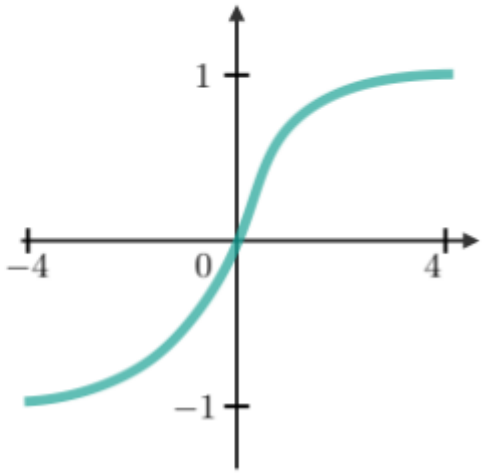
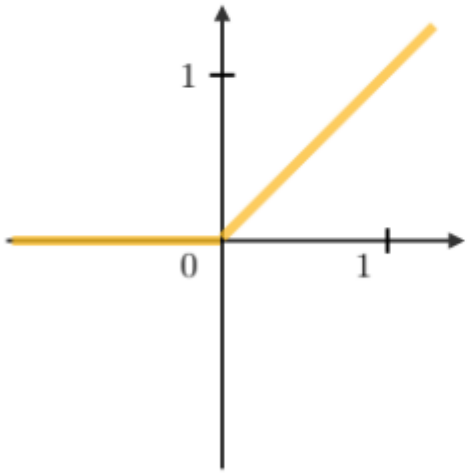
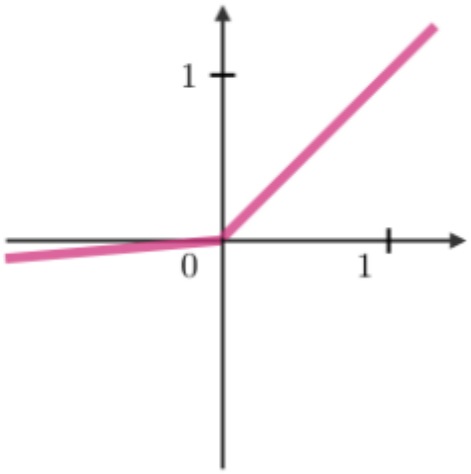
- Decide the network topology: Specify # of units in the input layer, # of hidden layers (if > 1), # of units in each hidden layer, and # of units in the output layer
- Normalize the input values for each attribute measured in the training tuples to [0.0—1.0]
- One input unit per domain value, each initialized to 0
- Output, if for classification and more than two classes, one output unit per class is used
- Once a network has been trained and its accuracy is unacceptable, repeat the training process with a different network topology or a different set of initial weights

A Multi-Layer Feed-Forward Neural Network



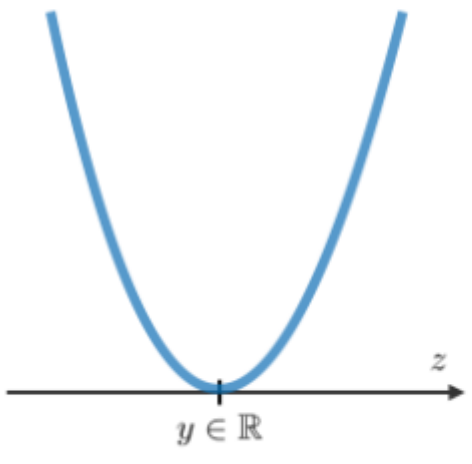
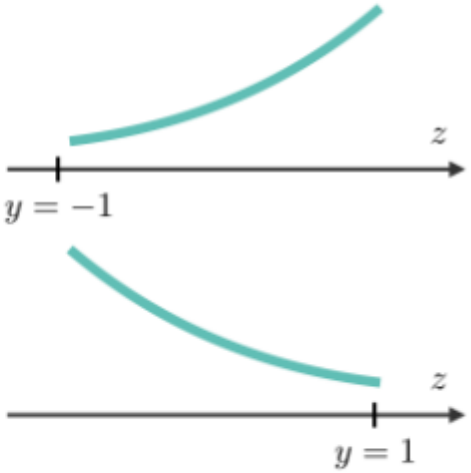
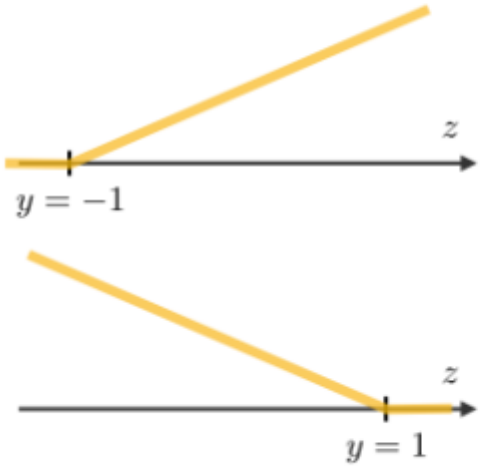
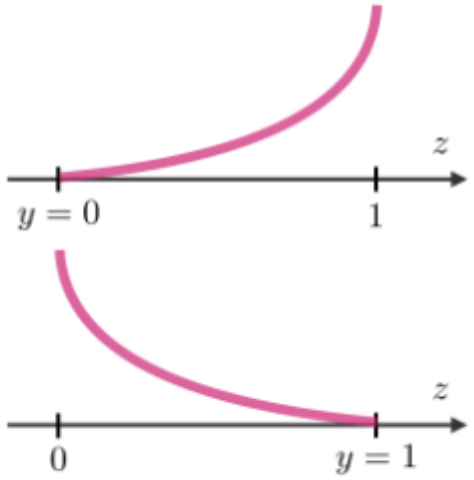
$$w_j^{(k+1)} = w_j^{(k)} + \lambda(y_i - \hat{y}_i^{(k)})x_{ij}$$

Hàm truyền (Acivation function)

Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$
			

https://stanford.edu/~shervine/teaching/cs-229/cheatsheet-deep-learning

Loss function

Least squared error	Logistic loss	Hinge loss	Cross-entropy
$\frac{1}{2}(y-z)^2$	$\log(1+\exp(-yz))$	$\max(0, 1-yz)$	$-\left[y\log(z)+(1-y)\log(1-z)\right]$
			
Linear regression	Logistic regression	SVM	Neural Network

https://stanford.edu/~shervine/teaching/cs-229/cheatsheet-supervised-learning

Tổng quan về hồi qui

- Mô hình hồi qui (regression model): mô hình mô tả mối liên kết (relationship) giữa một tập các biến dự báo (predictor variables/independent variables) và một hay nhiều đáp ứng (responses/dependent variables).
- Phân loại
  - Hồi qui tuyến tính (linear) và phi tuyến (nonlinear)
  - Hồi qui đơn biến (single) và đa biến (multiple)
  - Hồi qui có tham số (parametric), phi tham số (nonparametric), và tham số kết hợp (semiparametric)
  - Hồi qui đối xứng (symmetric) và bất đối xứng (asymmetric)

Hồi quy tuyến tính

- Thông thường có 3 phương pháp được sử dụng phổ biến để ước lượng mô hình hồi quy tuyến tính:
  - Ước lượng bình phương thông thường nhỏ nhất (Ordinary Least Squares - OLS)
  - Ước lượng bình phương tổng quát nhỏ nhất (GLS)
  - Ước lượng hợp lý cực đại (ML)

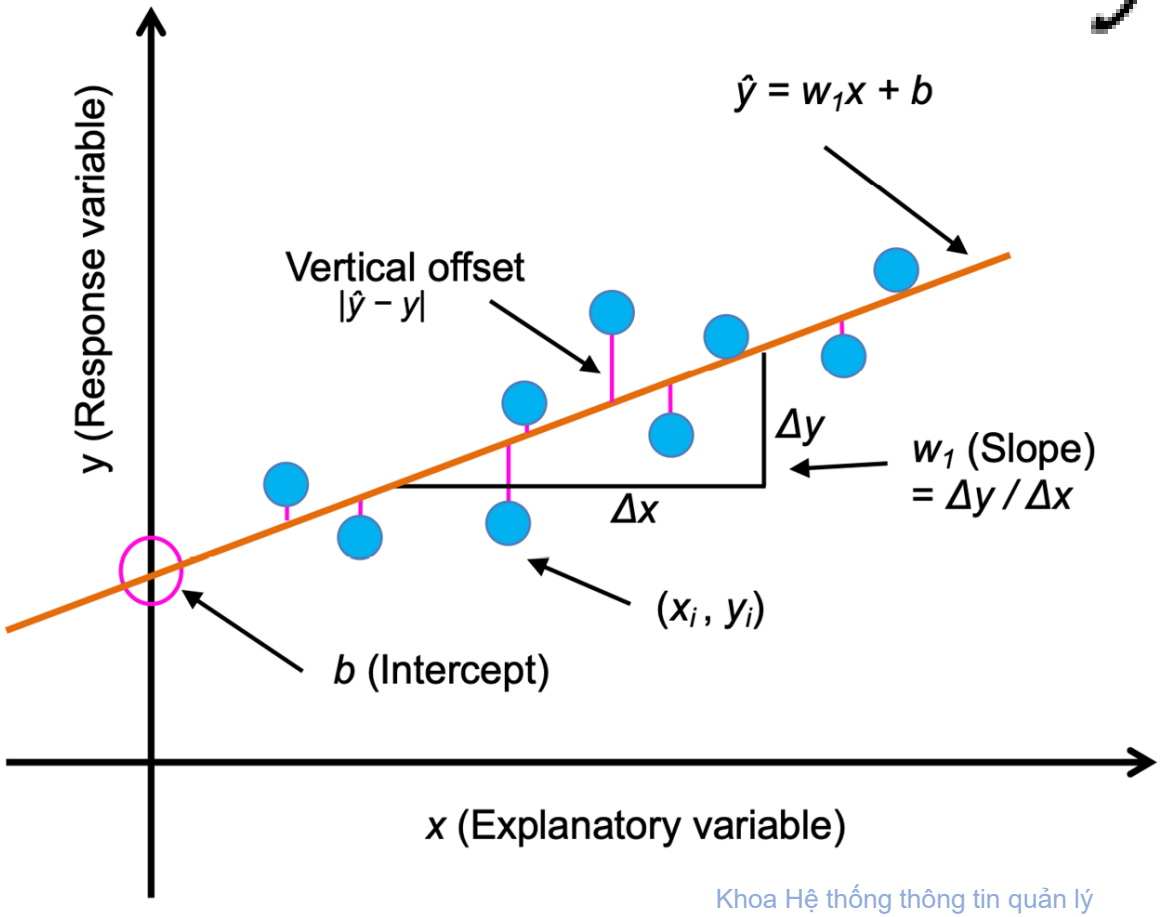
Hồi qui tuyến tính đơn biến

Cho N đối tượng đã được quan sát, mô hình hồi qui tuyến tính đơn biến được cho dưới dạng sau với ε_i dùng giữ phần biến thiên của đáp ứng Y không được giải thích từ X:

-Dạng đường thẳng $$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad i = 1, \ldots, N.$$

-Dạng parabola $$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon_i, \ i = 1, \ldots, N.$$

$$y = w_1 x + b$$


ŷ = w1x + b
Vertical offset
|ŷ − y|
y (Response variable)
Δy
w1 (Slope)
= Δy / Δx
Δx
(xi , yi)
b (Intercept)
x (Explanatory variable)

Hồi qui tuyến tính đơn biến: OLS

- Ước lượng bộ tham số số $\beta(\hat{\beta}_0, \hat{\beta}_1)$ để đạt được mô hình hồi qui tuyến tính đơn biến

$$y_i = \hat{\beta}_0 + \hat{\beta}_1 X_i + e_i.$$

Phần dư (residual)

$$e_i = y_i - \hat{y}_i$$

Tổng phần dư bình phương (sum of squared residuals) → tối thiểu hóa

$$SSE = \sum_{i=1}^{N} e_i^2.$$

Trị ước lượng của β

$$\hat{\beta}_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} \text{ and } \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Hồi qui tuyến tính đa biến: OLS

- Hồi qui tuyến tính đa biến: phân tích mối quan hệ giữa biến phụ thuộc (response/dependent variable) và hai hay nhiều biến độc lập (independent variables)

$$y = b_0 + b_1 x_{i1} + b_2 x_{i2} + \cdots + b_k x_{ik}$$

i = 1..n với n là số đối tượng đã quan sát
k = số biến độc lập (số thuộc tính/tiêu chí/yếu tố…)
Y = biến phụ thuộc
X = biến độc lập
b_0 = trị của Y khi X = 0
$b_{1..k}$ = trị của các hệ số hồi qui

Hồi qui tuyến tính đa biến: OLS

Trị ước lượng của Y $$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \ldots + b_k x_k$$

Trị ước lượng của bộ thông số b $$\mathbf{b} = \left(\mathbf{X}^T \mathbf{X}\right)^{-1} \mathbf{X}^T \mathbf{Y}$$

$$\mathbf{Y} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdots & x_{1,k} \\ 1 & x_{2,1} & x_{2,2} & \cdots & x_{2,k} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n,1} & x_{n,2} & \cdots & x_{n,k} \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_k \end{bmatrix}$$

Tập dữ liệu mẫu Toyota Corolla

Variable	Description
Price	Offer price in Euros
Age	Age in months as of August 2004
Kilometers	Accumulated kilometers on odometer
Fuel type	Fuel type (Petrol, Diesel, CNG)
HP	Horsepower
Metallic	Metallic color? (Yes = 1, No = 0)
Automatic	Automatic (Yes = 1, No = 0)
CC	Cylinder volume in cubic centimeters
Doors	Number of doors
QuartTax	Quarterly road tax in Euros
Weight	Weight in kilograms

Hồi quy tuyến tính với

Sử dụng package scikit-learn

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
data_path = '/content/drive/MyDrive/Colab Notebooks/dataset/ToyotaCorolla.csv'
df = pd.read_csv(data_path, index_col='Id')
car_df = df
car_df = car_df.iloc[0:1000]

predictors = ['Age_08_04', 'KM', 'HP', 'Weight']
outcome = 'Price'

vars = np.concatenate((predictors, [outcome]), axis = 0)

X = car_df[predictors]
y = car_df[outcome]
```

Hồi quy tuyến tính với

```
train_X, valid_X, train_y, valid_y = train_test_split(X, y, te
st_size=0.4, random_state=1)

car_lm = LinearRegression()
car_lm.fit(train_X, train_y)

yfit = car_lm.predict(train_X)
r2 = r2_score(train_y, yfit)

print("R^2 = ", r2)
print(pd.DataFrame({'Predictor': X.columns, 'coefficient': car
_lm.coef_}))
print('Intercetp: ', car_lm.intercept_)
```

Hồi quy tuyến tính với scikit-learn

```
R^2 =  0.8473832083037252
   Predictor  coefficient
0  Age_08_04  -141.669280
1         KM    -0.018081
2         HP    35.867180
3     Weight    14.896980
Intercetp:  0.8106596729776356
```

```
Regression statistics

                    Mean Error (ME) : -0.0000
         Root Mean Squared Error (RMSE) : 1444.2910
              Mean Absolute Error (MAE) : 1067.8798
            Mean Percentage Error (MPE) : -1.1617
Mean Absolute Percentage Error (MAPE) : 9.5524
```

Hồi quy tuyến tính với statsmodels

```
import pandas as pd
import numpy as np
import statsmodels.api as sm

data_path = '/content/drive/MyDrive/Colab Notebooks/dataset/ToyotaCorolla.csv'
df = pd.read_csv(data_path, index_col='Id')
car_df = df
car_df = car_df.iloc[0:1000]

predictors = ['Age_08_04', 'KM']
outcome = 'Price'

vars = np.concatenate((predictors, [outcome]), axis = 0)

X = car_df[predictors]
y = car_df[outcome]

train_X, valid_X, train_y, valid_y = train_test_split(X, y, test_size=0.4,
                                                      random_state=1)
mod = sm.OLS(train_y, train_X)

res = mod.fit()
print(res.summary())
```

Sử dụng package statsmodels

OLS Regression Results

Dep. Variable:	Price	R-squared (uncentered):	0.986
Model:	OLS	Adj. R-squared (uncentered):	0.986
Method:	Least Squares	F-statistic:	1.074e+04
Date:	Thu, 31 Mar 2022	Prob (F-statistic):	0.00
Time:	06:46:29	Log-Likelihood:	-5216.6
No. Observations:	600	AIC:	1.044e+04
Df Residuals:	596	BIC:	1.046e+04
Df Model:	4		
Covariance Type:	nonrobust		

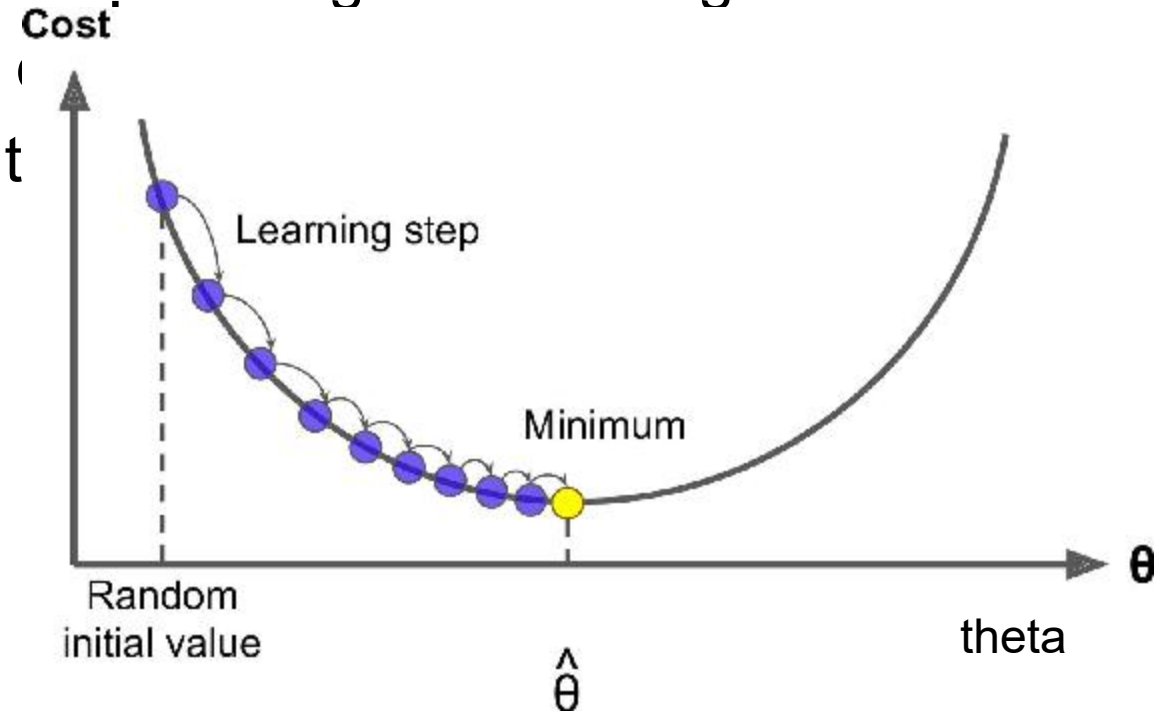
	coef	std err	t	P>|t|	[0.025	0.975]
Age_08_04	-141.6676	4.226	-33.523	0.000	-149.967	-133.368
KM	-0.0181	0.002	-8.243	0.000	-0.022	-0.014
HP	35.8678	4.023	8.915	0.000	27.966	43.769
Weight	14.8976	0.405	36.827	0.000	14.103	15.692

Omnibus:	64.293	Durbin-Watson:	1.960
Prob(Omnibus):	0.000	Jarque-Bera (JB):	430.027
Skew:	-0.098	Prob(JB):	4.18e-94
Kurtosis:	7.143	Cond. No.	5.77e+03

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.77e+03. This might indicate that there are
strong multicollinearity or other numerical problems.

Giải thuật học giảm độ dốc (Gradient Descent)

- Một hướng tiếp cận phổ biến để giải các bài toán tối ưu là xuất phát từ một điểm được cho là gần với nghiệm của bài toán, sau đó dùng phép lặp để tiến gần đến điểm cần tìm, cũng là đến khi đạo hàm gần với không.
- Gradient Descent và các biến thể của nó
  thuật sẽ tìm ra kết quả tối ưu sao cho tối t



Stochastic Gradient Descent với scikit-learn

class `sklearn.linear_model.`**`SGDRegressor`**(*loss='squared_error', *, penalty='l2', alpha=0.0001, l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001, shuffle=True, verbose=0, epsilon=0.1, random_state=None, learning_rate='invscaling', eta0=0.01, power_t=0.25, early_stopping=False, validation_fraction=0.1, n_iter_no_change=5, warm_start=False, average=False*)

Phải dùng dữ liệu đã chuẩn hóa

Hồi quy tuyến tính với GD

```
import numpy as np
from sklearn.linear_model import SGDRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
reg = make_pipeline(StandardScaler(),
                    SGDRegressor(max_iter=1000, tol=1e-
3))
reg.fit(train_X, train_y)
yfit = reg.predict(train_X)
r2 = r2_score(train_y, yfit)
print("R^2 = ", r2)
```

Hồi quy logic (Logistic Regression)

- Nếu với hồi quy tuyến tính với đầu ra là **giá trị thực (liên tục)** thì đầu ra của hồi quy logic là **nhị phân**
- Ví dụ: để chuẩn đoán kết quả là có bệnh hay không, giao dịch tài chính có phải gian lận hay không,….

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q)}}.$$

logistic response function

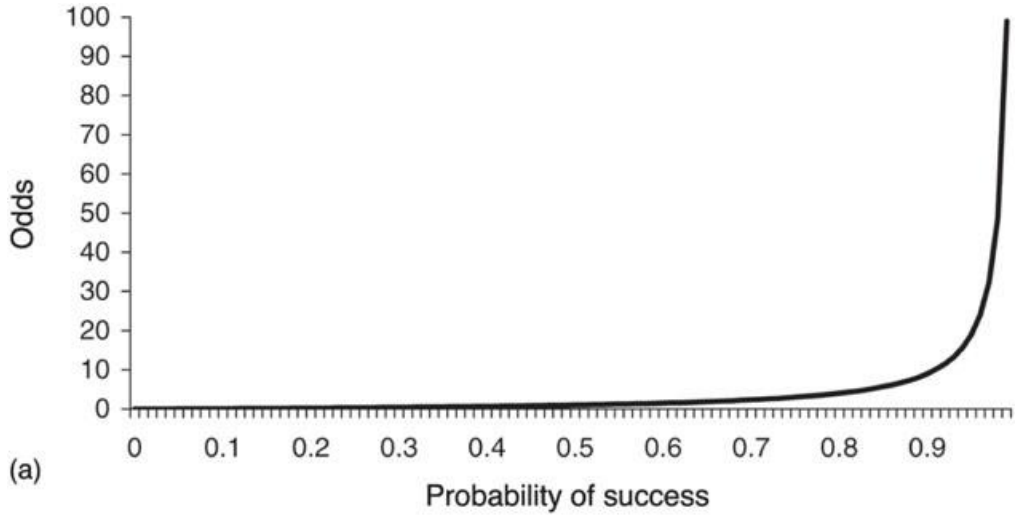
$$\text{Odds}(Y = 1) = \frac{p}{1 - p}.$$

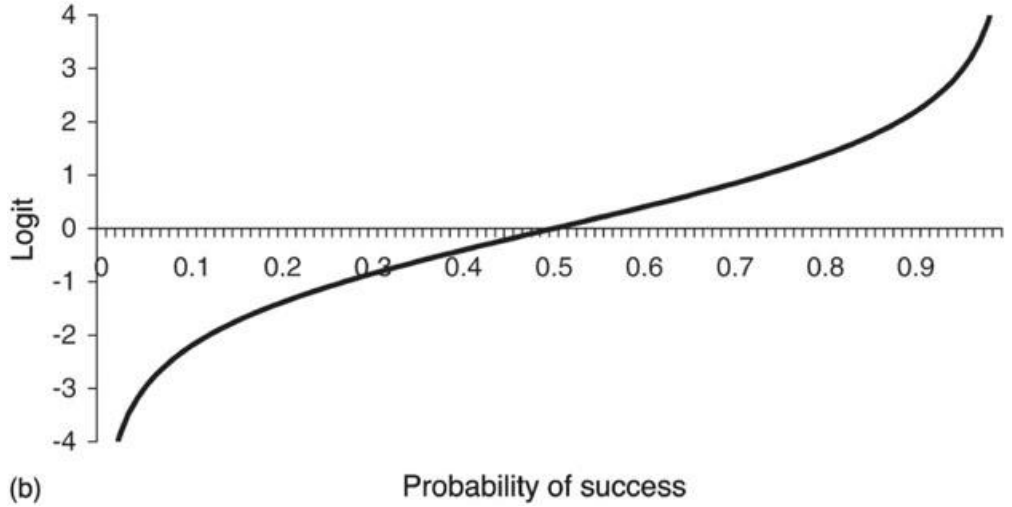
Tỉ số giữa xác xuất thuộc về lớp 1 và xác xuất thuộc về lớp 0

$$\text{Odds}(Y = 1) = e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q}.$$

$$\log(\text{odds}) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q.$$

Hồi quy logic (Logistic Regression)





Đánh giá mô hình hồi quy (nhãn liên tục)

R-Squared (R2), Adjusted R-Squared

Mean absolute error (MAE)

$$\text{MAE} = \frac{1}{\text{N}}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

Mean Squared Error (MSE)

$$\text{MSE} = \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{N}$$

Root mean square error (RMSE)

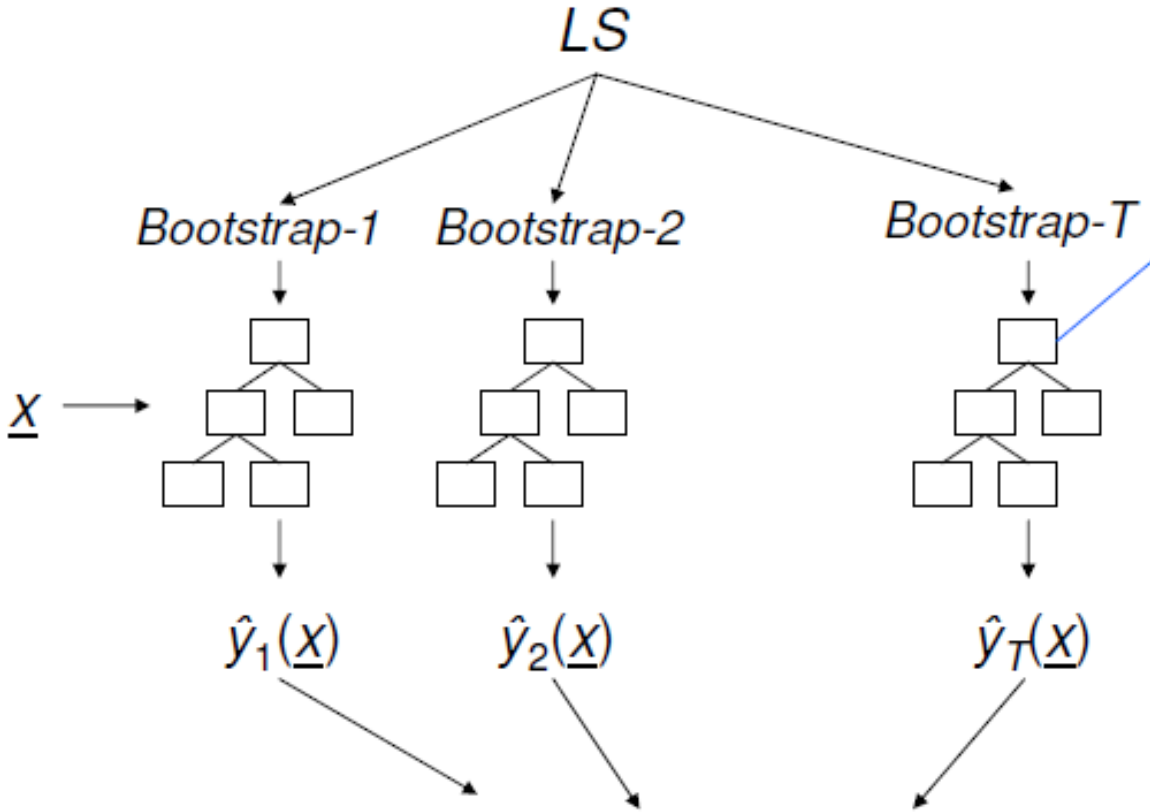
$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{N}}$$

Mean absolute percentage error (MAPE)

$$\text{MAPE} = \frac{\sum_{i=1}^{N}\frac{|y_i - \hat{y}_i|}{y_i}}{N}$$

Bayesian Information Criteria (BIC)

Random Forest - Rừng cây ngẫu nhiên (RF)



Nút trong: chọn ngẫu nhiên n' thuộc tính và tính toán phân hoạch tốt nhất dựa trên n' thuộc tính này

Chú ý: xây dựng cây không cắt nhánh

hồi quy : $\hat{y}(\underline{x}) = (\hat{y}_1(\underline{x})+\hat{y}_2(\underline{x})+\ldots+\hat{y}_T(\underline{x}))/\mathrm{T}$

phân loại : $\hat{y}(\underline{x})$ = bình chọn số đông $\{\hat{y}_1(\underline{x}),\ldots,\hat{y}_T(\underline{x})\}$

Tóm tắt

- Phân lớp:
  - Hồi quy logit (Logistic regression)
  - Decision Tree Classifier
  - Support Vector Classification (SVC)
  - Nearest Neighbors Classification (KNeighborsClassifier)
  - Stochastic Gradient Descent Classification (SGDClassifier)
  - Gaussian Naive Bayes, Categorical Naive Bayes, Multinomial Naive Bayes
  - Neural network - Classification
- Hồi quy:
  - Ordinary Least Squares (LinearRegression)
  - Decision Tree Regressor
  - Support Vector Regression (SVR)
  - Stochastic Gradient Descent Regression (SGDRegressor)
  - Nearest Neighbors Regression (KNeighborsRegressor)
  - Neural network - Regression



THANK YOU FOR WATCHING!